

System Architecture Document

Project: NS Engineering Website Migration

Intern: Ritika Kunwar

Reviewed By: Shobhit Tripathi (Mentor)

Status: Updated / Dev-Ready

Current Date: May 22, 2026

TABLE OF CONTENT

Contents

1. Executive Summary & Objective	5
2. System Diagrams.....	5
2.1 Old System (GitHub Pages)	5
2.2 Multi-Environment Architecture (Cloudflare Pages).....	6
3. Core Architectural Principles (For Dev)	7
3.1 Immutable Artifact Promotion	7
3.2 Separation of Concerns	7
4. Environment & Branch Specifications	7
4.1 GitHub Environment Configuration.....	8
4.2 Dev Secrets.....	8
5. Active Pipeline Configuration (deploy-dev.yml)	9
5.1 Workflow Trigger	9
5.2 Permissions	9
5.3 Global Environment Variables	9
5.4 Jobs.....	9
5.4.1 Job 1: sync-assets (Google Drive → R2 Sync).....	10
5.4.2 Job 2: build (Next.js Build Step)	11
5.4.3. Job 3: deploy (Cloudflare Pages Deployment).....	12
6. Operational Validation	17
6.1 UI Pipeline Verification	17
6.2 DNS Configuration	18
6.3 Host Domain Verification.....	19
7. Email Worker Architecture	19
7.1 Domain	19
7.2 Client Communication.....	19
7.3 Dev Environment Mapping.....	19
7.4 Complete System Flow.....	21
8. Conclusion	22

LIST OF FIGURES

Figure 1: GitHub Pages Deployment.....	6
Figure 2: Dev Setup.....	6
Figure 3:Dev-Email Setup.....	21

LIST OF TABLES

Table 1: Branch Mapping for each enviroment	8
Table 2: Dev secrets	8
Table 3: Dev Environment Mapping	20

1. Executive Summary & Objective

This document outlines the architectural transition of the **NS Engineering Website** from a single-environment GitHub Pages deployment to a multi-environment, highly secure infrastructure hosted on **Cloudflare Pages**.

The main reason for this migration is to make the deployment process safer and more controlled.

With the old setup using GitHub Pages, changes could go directly to production automatically, which increases the risk of mistakes going live without proper checking.

By moving to Cloudflare, We can:

- Keep development, testing, and production environments completely separate.
- Add approval steps so a person reviews changes before they go live.
- Prevent unreviewed or unsafe updates from reaching real users.

2. System Diagrams

2.1 Old System (GitHub Pages)

Previously, whenever code pushed to the repository, the website was automatically rebuilt using live cloud data and instantly updated on the real production website.

This meant there were no protection steps such as testing, approvals, or separate environments before changes went live. As a result, even a small mistake could immediately affect real users and the live website.

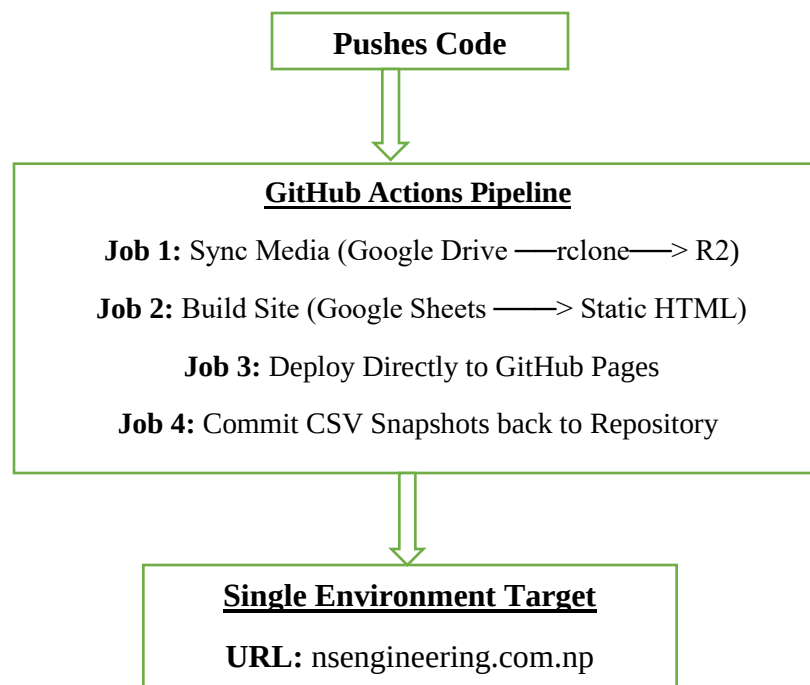


Figure 1: GitHub Pages Deployment

2.2 Multi-Environment Architecture (Cloudflare Pages)

The new system follows a “**Build Once, Promote Many**” approach. Which means the code is built only one time, and the same verified version is moved step-by-step through separate environments like development, staging, and production.

Instead of rebuilding the code again in each environment, the already-tested build is promoted forward only after approval checks using GitHub Environment gates.

For Dev Environment:

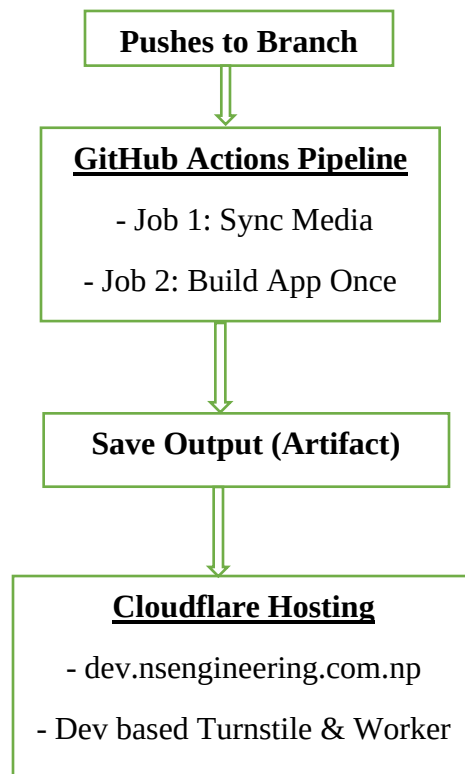


Figure 2: Dev Setup

3. Core Architectural Principles (For Dev)

3.1 Immutable Artifact Promotion

The website's static files (`./out`) are created only once by running **`npm run build:cloud`**, which builds the site using cloud-based data sources.

After the build is completed, this generated folder is saved as a separate pipeline artifact.

Later, the deployment step simply downloads and deploys this same saved artifact instead of rebuilding the project again.

3.2 Separation of Concerns

We configured Cloudflare Pages to use Direct Uploads through the Wrangler CLI (**`cloudflare/wrangler-action@v3`**) instead of using Cloudflare's automatic GitHub integration.

Because:

To keep responsibilities separate:

- GitHub Actions handles the entire CI/CD pipeline logic such as:
 - building,
 - testing,
 - approvals,
 - and deployment flow.
- Cloudflare only handles hosting the already-built website files at the edge.

This will avoid depending too heavily on one platform.

Because the deployment process is controlled inside GitHub Actions, and we can later move hosting from Cloudflare to platforms like Amazon Web Services or Vercel without redesigning the whole pipeline.

4. Environment & Branch Specifications

Environment	Trigger Condition	Protection Rules	Custom Domain Mapping	Target Cloudflare Project
Development (dev)	Every push to feature/*.	None. Automated Deployment.	dev.nsengineering.com.np	nsengineering-dev
Staging (stage)	Promoted from	Required Review..	stage.nsengineering.com.np	nsengineering-stage

Environment	Trigger Condition	Protection Rules	Custom Domain Mapping	Target Cloudflare Project
	successful dev baseline.			
Production (prod)	Promoted from verified staging state.	Required Review.+ 10-minute minimum mandatory wait time.	nsengineering.com.np	nsengineering-prod

Table 1: Branch Mapping for each environment

4.1 GitHub Environment Configuration

- **Environment Name:** dev
- **Protection Rules:** None

4.2 Dev Secrets

These secrets are isolated to the dev environment context:

Secret Key	Description
CLOUDFLARE_ACCOUNT_ID	Directs the Wrangler API to our target account.
CLOUDFLARE_API_TOKEN	Grants write permission scoped <i>only</i> to the dev Pages project.
CLOUDFLARE_PROJECT_NAME	Targets the specific project: nsengineering-dev.
NEXT_PUBLIC_EMAIL_WORKER_URL	Endpoint routing contact forms to the dev Cloudflare Worker.
NEXT_PUBLIC_TURNSTILE_SITE_KEY	Public key rendering dev bot verification.
NEXT_PUBLIC_R2_BASE_URL	Points image assets to the development R2 container.
R2_ACCESS_KEY_ID	Controls authorized media reads and writes.
R2_SECRET_ACCESS_KEY	used to securely authenticate with the R2 storage account.
R2_BUCKET_NAME	To specify specific R2 storage bucket
R2_ACCOUNT_ID	unique identifier for the Cloudflare account that owns the R2 storage

Table 2: Dev secrets

5. Active Pipeline Configuration (deploy-dev.yml)

GitHub Actions workflow file below contains the execution steps currently deployed for the dev stage.

This GitHub Actions workflow is an automated CI/CD pipeline that pulls assets from Google Drive, syncs them to Cloudflare R2, builds a Next.js site, and finally deploys it to Cloudflare Pages using Wrangler. It is structured into three main jobs that run in sequence: sync-assets, build, and deploy.

5.1 Workflow Trigger

The workflow starts with:

- **Trigger (on)**
 - Runs automatically on pushes to the configured branch
 - Can also be triggered manually using workflow_dispatch

So, every time we push code to that feature branch, the pipeline runs automatically.

5.2 Permissions

- **permissions: contents: write**
 - Allows the workflow to modify repository contents if needed.
- **concurrency group: cloudflare-pages-dev**
 - Ensures only one deployment runs at a time for the dev environment.
 - cancel-in-progress: false means new runs will wait instead of canceling ongoing ones.

This will prevent deployment conflicts.

5.3 Global Environment Variables

These are shared across all jobs:

- CONTENT_SOURCE_MODE: sheets → content is coming from Google Sheets
- NEXT_PUBLIC_R2_BASE_URL → base URL for assets stored in R2
- GOOGLE_SHEET_ID → identifies the content source
- NEXT_PUBLIC_TURNSTILE_SITE_KEY → bot protection (Cloudflare Turnstile)
- NEXT_PUBLIC_EMAIL_WORKER_URL → backend email service endpoint

These values are injected securely using GitHub Secrets.

5.4 Jobs

5.4.1 Job 1: sync-assets (Google Drive → R2 Sync)

This job runs first and prepares media assets.

Purpose

It syncs files from Google Drive into Cloudflare R2 storage using rclone.

Steps Explained

1. Checkout repository

Pulls code into the runner so scripts and configs are available.

2. Install rclone

Installs rclone, a command-line tool used for syncing cloud storage systems.

3. Configure Google Drive remote

Creates a rclone.conf file with a **service account authentication JSON**.

This defines:

- Google Drive connection (gdrive)
- Permissions (read-only access)
- Root folder ID

This is how the workflow securely accesses Drive files.

4. Configure Cloudflare R2 remote

Adds configuration for R2 storage:

- Uses S3-compatible API
- Uses access keys + account ID
- Sets endpoint like:
`https://<account-id>.r2.cloudflarestorage.com`

This allows rclone to upload files into R2.

5. Sync media files

Runs the core command:

- Copies files from gdrive: → r2:<bucket>
- Includes only:
 - jpg, jpeg, png, webp, pdf
- Excludes CSV files

So only media assets are transferred.

6. Verify upload

Lists files in R2 bucket to confirm upload succeeded.

5.4.2 Job 2: build (Next.js Build Step)

This job depends on sync-assets, so it runs after assets are ready.

Purpose

Builds the website using Next.js.

Steps Explained

1. Checkout code

Pulls latest repository state.

2. Setup Node.js

Installs Node.js 20 and enables npm caching for faster builds.

3. Install dependencies

Runs:

```
npm ci
```

This ensures a clean, reproducible dependency install.

4. Create Google credentials file

This allows the app to access Google APIs during build time.

5. Build Next.js app

Runs:

```
npm run build:cloud
```

This will:

- Fetches data from Google Sheets
- Generates static pages
- Outputs final site into /out

6. Upload build artifact

Stores /out as a GitHub Actions artifact named:

site-build

This artifact is passed to the next job.

5.4.3. Job 3: deploy (Cloudflare Pages Deployment)

This is the final step that publishes your site.

It depends on both:

- build
- sync-assets

Purpose

Deploys the static site to Cloudflare Wrangler which pushes it to Cloudflare Pages.

Steps Explained

1. Download build artifact

Retrieves the /out folder created in the build job.

2. Setup Node.js

Ensures Node environment exists for Wrangler.

3. Install Wrangler

Installs Cloudflare CLI:

```
npm install -g wrangler
```

4. Deploy to Cloudflare Pages

Runs:

```
wrangler pages deploy ./out --project-name=...
```

This:

- Uploads static site files
- Deploys them to our Cloudflare Pages project
- Makes it live in dev environment

Authentication uses:

- API Token
- Account ID

5. Deployment complete message

Just logs success confirmation.

YAML:

name: Deploy to Cloudflare Pages (Dev)

on:

push:

branches: [feature/ritika/Goal3]

workflow_dispatch:

permissions:

contents: write

concurrency:

group: cloudflare-pages-dev

cancel-in-progress: false

env:

CONTENT_SOURCE_MODE: sheets

NEXT_PUBLIC_R2_BASE_URL: \${ secrets.NEXT_PUBLIC_R2_BASE_URL }

GOOGLE_SHEET_ID: \${ secrets.GOOGLE_SHEET_ID }

NEXT_PUBLIC_TURNSTILE_SITE_KEY: \${ secrets.NEXT_PUBLIC_TURNSTILE_SITE_KEY }

NEXT_PUBLIC_EMAIL_WORKER_URL: \${ secrets.NEXT_PUBLIC_EMAIL_WORKER_URL }

jobs:

sync-assets:

```

name: Sync Google Drive → R2
runs-on: ubuntu-latest

# Enables ENVIRONMENT secrets
environment: dev

env:
  R2_BUCKET_NAME: ${ secrets.R2_BUCKET_NAME }
  R2_ACCESS_KEY_ID: ${ secrets.R2_ACCESS_KEY_ID }
  R2_SECRET_ACCESS_KEY: ${ secrets.R2_SECRET_ACCESS_KEY }
  R2_ACCOUNT_ID: ${ secrets.R2_ACCOUNT_ID }

steps:
  - name: Checkout repository
    uses: actions/checkout@v4

  - name: Install rclone
    run: |
      set -euo pipefail
      curl https://rclone.org/install.sh | sudo bash
      rclone version

  - name: Configure Google Drive remote
    run: |
      set -euo pipefail
      mkdir -p ~/.config/rclone
      cat > ~/.config/rclone/rclone.conf <<EOF
      [gdrive]
      type = drive
      scope = drive.readonly
      service_account_credentials =
      {"type": "service_account", "project_id": "${ secrets.GDRIVE_PROJECT_ID
      }", "private_key_id": "${ secrets.GDRIVE_PRIVATE_KEY_ID }", "private_key": "${
      secrets.GDRIVE_PRIVATE_KEY }", "client_email": "${
      secrets.GDRIVE_SERVICE_ACCOUNT_EMAIL }", "client_id": "${
      secrets.GDRIVE_CLIENT_ID
      }", "auth_uri": "https://accounts.google.com/o/oauth2/auth", "token_uri": "https://o
      auth2.googleapis.com/token", "auth_provider_x509_cert_url": "https://www.googleapis
      .com/oauth2/v1/certs", "client_x509_cert_url": "https://www.googleapis.com/robot/v1
      /metadata/x509/${ secrets.GDRIVE_SERVICE_ACCOUNT_EMAIL }"}
      root_folder_id = ${ secrets.GDRIVE_ROOT_FOLDER_ID }
      EOF

  - name: Configure Cloudflare R2 remote
    run: |

```

```

set -euo pipefail
cat >> ~/.config/rclone/rclone.conf <<EOF

[r2]
type = s3
provider = Cloudflare
access_key_id = $R2_ACCESS_KEY_ID
secret_access_key = $R2_SECRET_ACCESS_KEY
endpoint = https://$R2_ACCOUNT_ID.r2.cloudflarestorage.com
acl = public-read
EOF

- name: Sync media files to R2
  run: |
    set -euo pipefail
    echo "🌀 Syncing media files from Google Drive to R2..."

    rclone sync gdrive: r2:${R2_BUCKET_NAME} \
      --filter "- *.csv" \
      --filter "+ *.jpg" \
      --filter "+ *.jpeg" \
      --filter "+ *.png" \
      --filter "+ *.webp" \
      --filter "+ *.pdf" \
      --progress \
      --checksum \
      --log-level INFO \
      --stats 30s
    echo "✅ Asset sync completed successfully"
- name: Verify uploaded R2 files
  run: |
    echo "📦 Listing R2 bucket files..."
    rclone ls r2:${R2_BUCKET_NAME} | head -50

build:
  name: Build Next.js Site (Dev)
  runs-on: ubuntu-latest

  # Enables ENVIRONMENT secrets
  environment: dev

  needs: sync-assets

  env:
    R2_ACCOUNT_ID: ${ secrets.R2_ACCOUNT_ID }

```

```

R2_ACCESS_KEY_ID: ${ secrets.R2_ACCESS_KEY_ID }}
R2_SECRET_ACCESS_KEY: ${ secrets.R2_SECRET_ACCESS_KEY }}
R2_BUCKET_NAME: ${ secrets.R2_BUCKET_NAME }}
NEXT_PUBLIC_R2_BASE_URL: ${ secrets.NEXT_PUBLIC_R2_BASE_URL }}

# Repository secrets
GOOGLE_SHEET_ID: ${ secrets.GOOGLE_SHEET_ID }}

steps:
  - name: Checkout code
    uses: actions/checkout@v4

  - name: Setup Node.js
    uses: actions/setup-node@v4
    with:
      node-version: "20"
      cache: npm

  - name: Install dependencies
    run: npm ci

  - name: Create Google credentials file
    run: echo '${ secrets.GOOGLE_CREDENTIALS_JSON }' > google-credentials.json

  - name: Build Next.js app
    env:
      GOOGLE_APPLICATION_CREDENTIALS: ./google-credentials.json
    run: npm run build:cloud

  - name: Upload build artifact
    uses: actions/upload-artifact@v4
    with:
      name: site-build
      path: ./out

deploy:
  name: Deploy to Cloudflare Pages (Dev via Wrangler)
  runs-on: ubuntu-latest
  environment: dev
  needs:
    - build
    - sync-assets

steps:

```

```

- name: Download build artifact
  uses: actions/download-artifact@v4
  with:
    name: site-build
    path: ./out

- name: Setup Node.js
  uses: actions/setup-node@v4
  with:
    node-version: "20"

- name: Install Wrangler
  run: npm install -g wrangler

- name: Deploy to Cloudflare Pages
  env:
    CLOUDFLARE_API_TOKEN: ${ secrets.CLOUDFLARE_API_TOKEN }
    CLOUDFLARE_ACCOUNT_ID: ${ secrets.CLOUDFLARE_ACCOUNT_ID }
  run: |
    wrangler pages deploy ./out \
      --project-name=${ secrets.CLOUDFLARE_PROJECT_NAME }

- name: Deployment complete
  run: echo "🚀 Dev deployment successful via Wrangler"

```

6. Operational Validation

These are performed to make sure the dev deployment is working correctly and safely after setup.

6.1 UI Pipeline Verification

1. Open the GitHub repository settings.
2. Go to **Settings** → **Environments**.
3. Check the **dev** environment and confirm:
 - there are no deployment restriction rules,
 - and all required environment secrets are configured correctly.

Next:

4. Push a small change to the active branch (feature/*).
5. Confirm that the GitHub Actions pipeline starts automatically.
6. Verify that all workflow stages complete successfully:
 - sync-assets
 - build

- deploy-dev

A successful run confirms the CI/CD pipeline is functioning correctly.

6.2 DNS Configuration

Purpose

The development deployment must run on a separate subdomain so testing activities do not affect the live production website.

DNS Changes Required

The following DNS configuration must be added inside the Cloudflare DNS dashboard for the domain:

- Cloudflare

Required DNS Record

Type	Name	Target	Proxy Status
CNAME	dev	Dev.nsengineering.com.np	Proxied

Explanation of the Configuration

1. CNAME Record

A CNAME record connects the custom development subdomain:

- dev.nsengineering.com.np

2. Proxied Status

The DNS record must remain **Proxied** because this allows Cloudflare to provide:

- HTTPS/SSL support
- CDN caching
- DDoS protection

6.3 Host Domain Verification

1. Open a browser and visit:
 - `dev.nsengineering.com.np`
2. Check that:
 - the website loads properly.

This confirms that the frontend is correctly connected to the development R2 storage endpoint using `NEXT_PUBLIC_R2_BASE_URL`.

7. Email Worker Architecture

The email system is designed as a separately using Cloudflare Workers. Even though all environments use the same codebase, each environment runs as its own isolated worker instance.

7.1 Domain

The development Turnstile setup is designed to only accepts requests coming from:

- `dev.nsengineering.com.np`

The secret verification key (`TURNSTILE_SECRET_KEY`) is securely stored inside the Wrangler secret store using:

```
wrangler secret put TURNSTILE_SECRET_KEY --env dev.
```

The secret verification key (`TURNSTILE_SECRET_KEY`) is also securely stored inside the Github environment secret.

7.2 Client Communication

The development worker is connected only to a testing inbox.

So, when someone submits the contact form on the development website:

- emails go to:
configuredEmail@domain.com
- instead of the real client inbox.

7.3 Dev Environment Mapping

Configuration Layer	Secrets	Purpose
GitHub Environment	NEXT_PUBLIC_EMAIL_WORKER_URL	Sends frontend from the email worker
GitHub Environment	NEXT_PUBLIC_TURNSTILE_SITE_KEY	Loads the Turnstile widget for the dev domain
Wrangler Store Secret	TURNSTILE_SECRET_KEY	Allows backend validation with Cloudflare Turnstile
Wrangler Store Secret	RESEND_API_KEY	to send emails from applications, websites, forms, and backend servers.

Table 3: Dev Environment Mapping

7.4 Complete System Flow

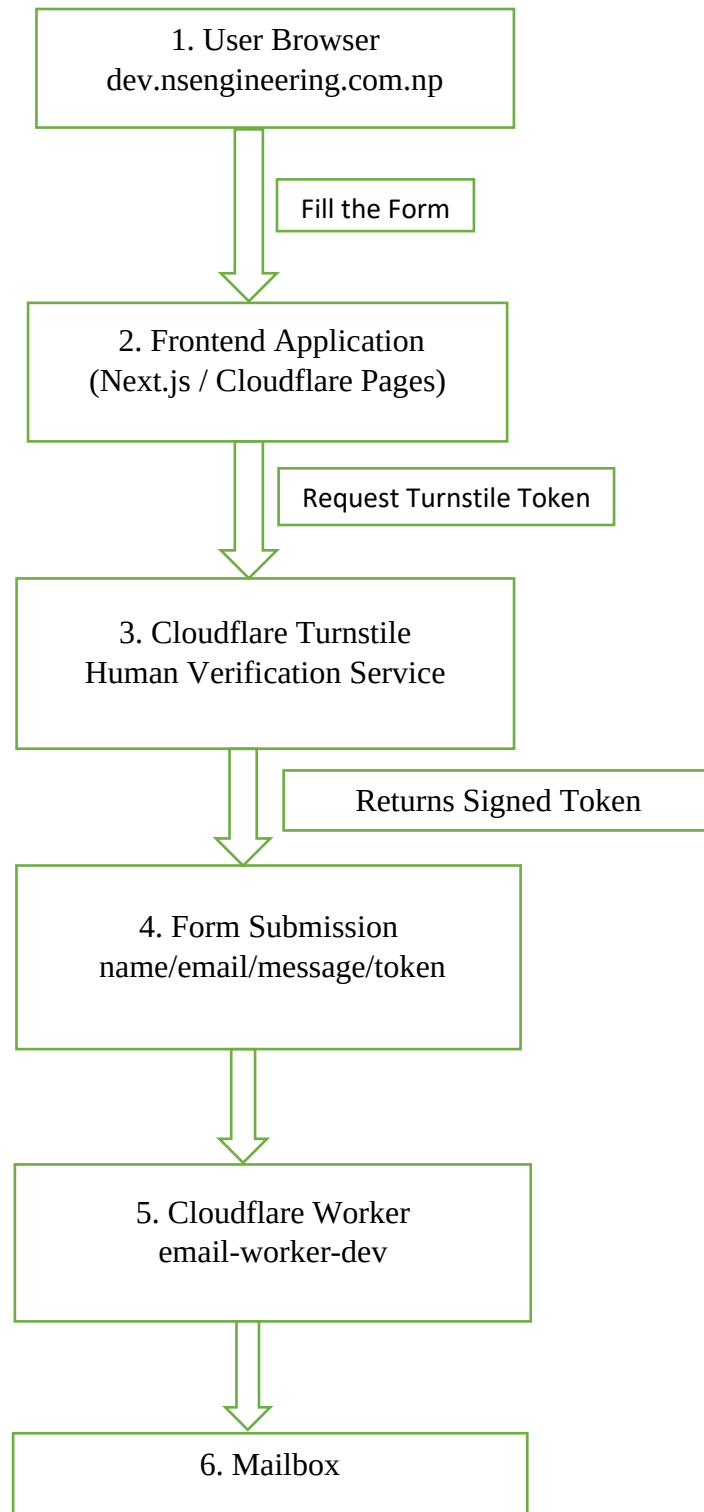


Figure 3: Dev-Email Setup

8. Conclusion

The migration of the NS Engineering Website (Development Environment) from GitHub Pages to Cloudflare Pages and Workers has been completed successfully and is now fully tested and live.

This new setup replaces the old single-environment system with a more secure, reliable, and isolated development environment.